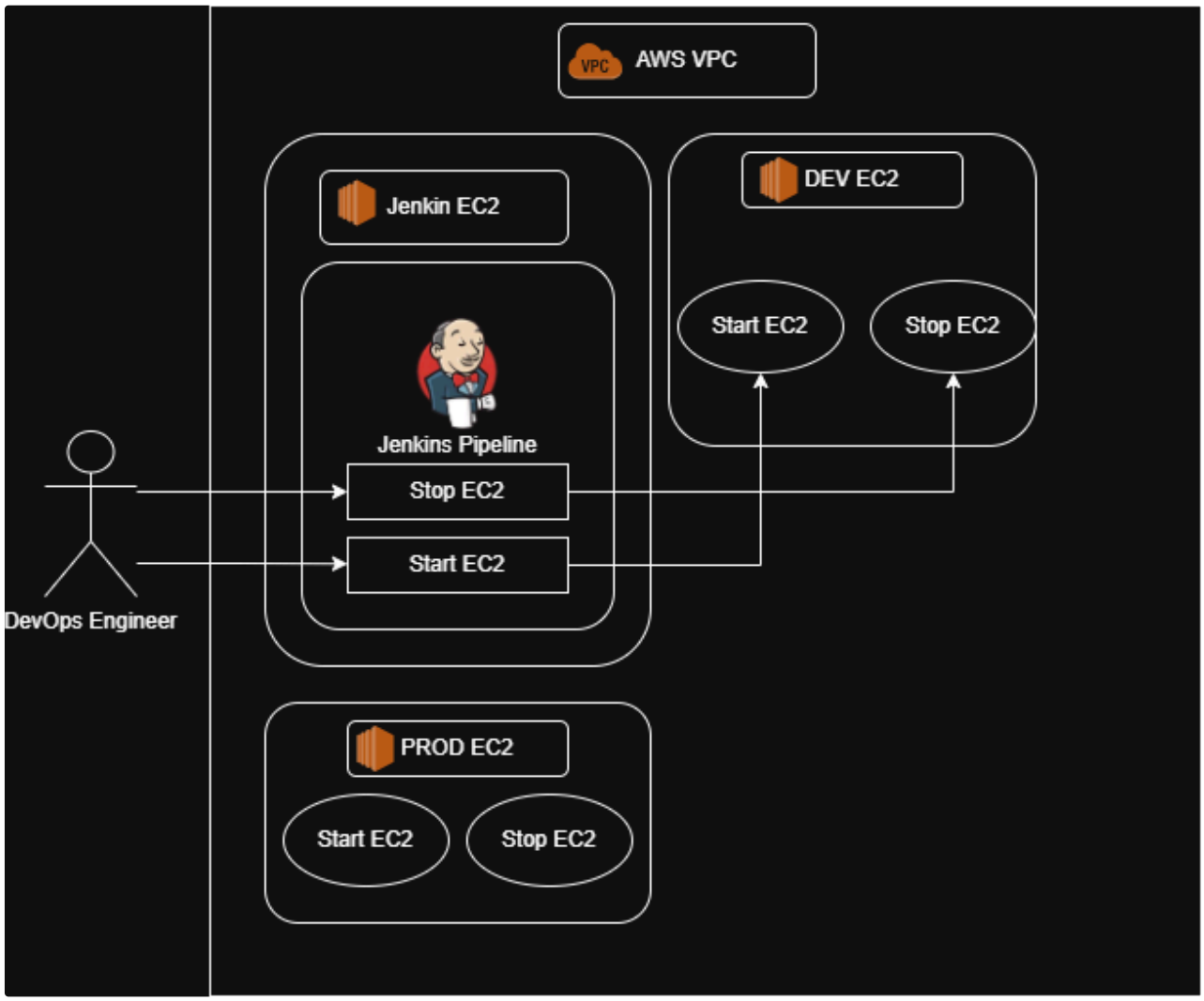
Task-2 Documentation (Automating EC2 Start/Stop with Jenkins Pipelines (Tag-Based))

1. Project Overview

This project demonstrates the automation of AWS resource states (Start/Stop) using Jenkins Pipelines. By utilizing the AWS CLI and resource tagging, we implemented a system that manages environment costs by ensuring resources like EC2 and ECS Fargate only run when required.

---

2. Infrastructure Setup & Security

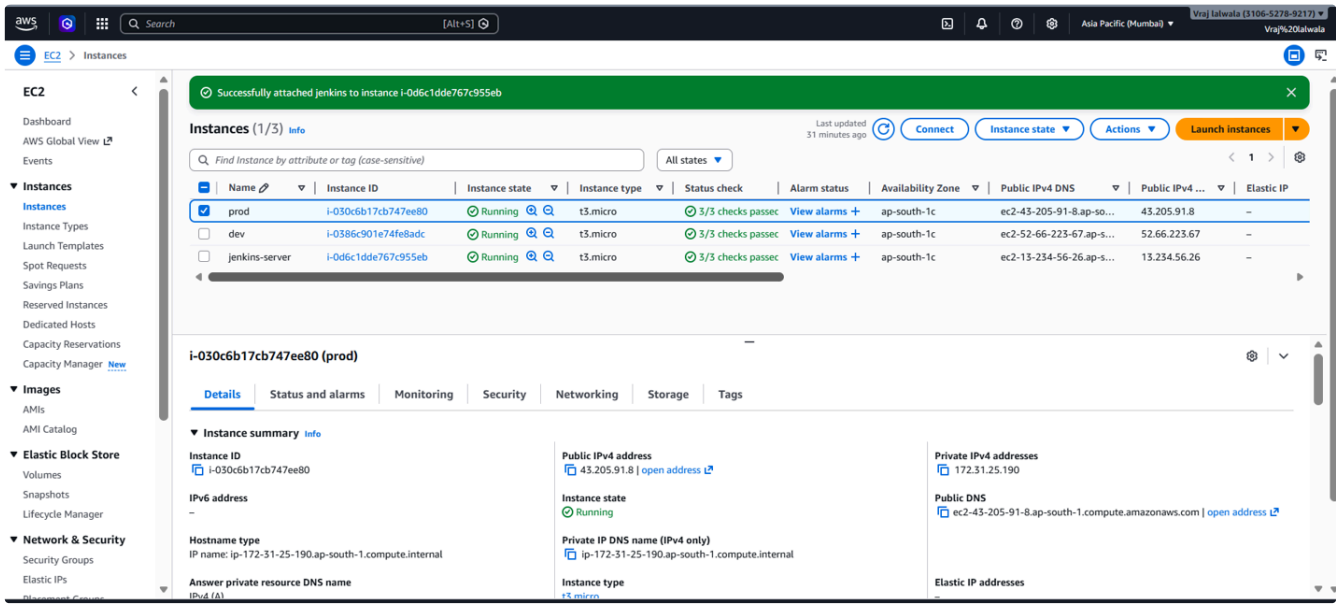


2.1 EC2 Instance Provisioning

Three EC2 instances were provisioned in the ap-south-1 region:

- dev: Managed instance for development tasks.

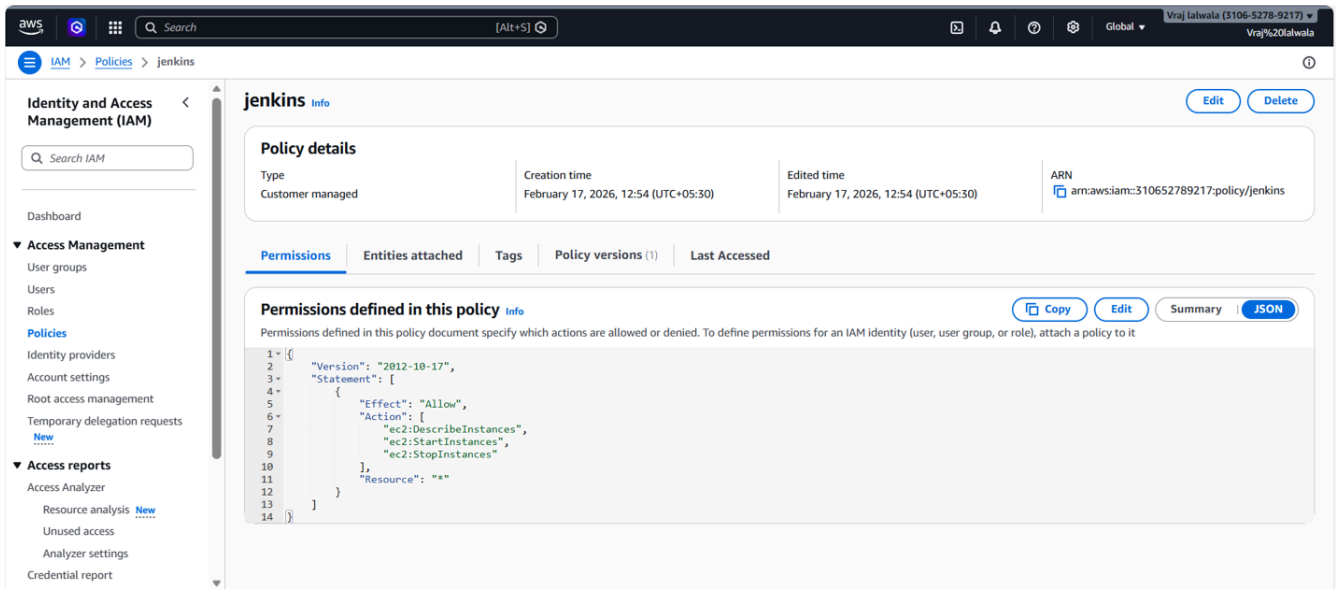
- prod: Production instance.
- jenkins-server: Hosted the Jenkins automation engine.



2.2 IAM Security Configuration

A custom IAM policy named `jenkins` was created to grant the Jenkins server the necessary permissions while maintaining the Principle of Least Privilege:

- **Permissions:** `ec2:DescribeInstances`, `ec2:StartInstances`, and `ec2:StopInstances`.
- **Policy ARN:** `arn:aws:iam::310652789217:policy/jenkins`.



---

3. Jenkins Pipeline Implementation (EC2)

3.1 Pipeline: Start-Dev-EC2

Purpose

Start the Dev EC2 instance using AWS CLI from Jenkins.

Pipeline Script

```
pipeline {
    agent any
    stages {
        stage('Start Dev EC2') {
            steps {
                sh '''
                aws ec2 start-instances --instance-ids i-0011ae80853c1a8c0
                aws ec2 wait instance-running --instance-ids i-0011ae80853c1a8c0
                echo "Dev EC2 started successfully"
                '''
            }
        }
    }
}

```

Execution Verification

- Jenkins console output shows instance state transitioning from stopped to running.
- AWS EC2 Console confirms instance state as Running.

3.2 Pipeline: Stop-Dev-EC2

Purpose

Stop the Dev EC2 instance using Jenkins.

Pipeline Script

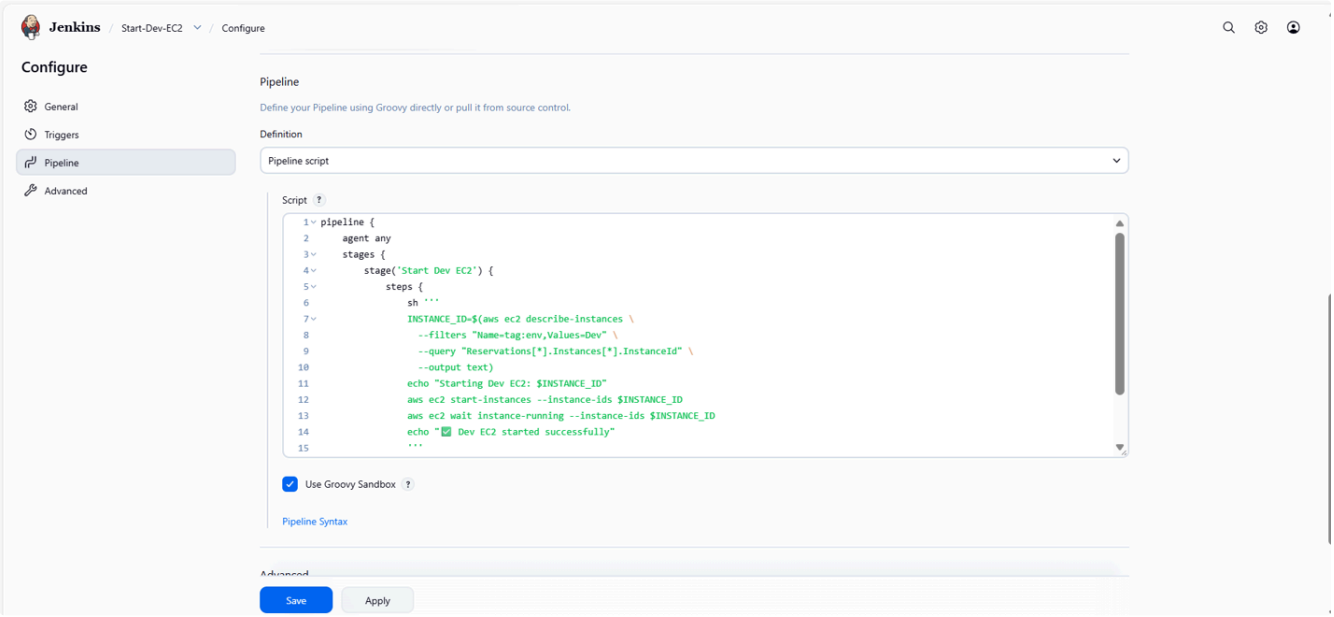
```
pipeline {
    agent any
    stages {
        stage('Stop Dev EC2') {
            steps {
                sh '''
                aws ec2 stop-instances --instance-ids i-0011ae80853c1a8c0
                aws ec2 wait instance-stopped --instance-ids i-0011ae80853c1a8c0
                echo "Dev EC2 stopped successfully"
                '''
            }
        }
    }
}

```

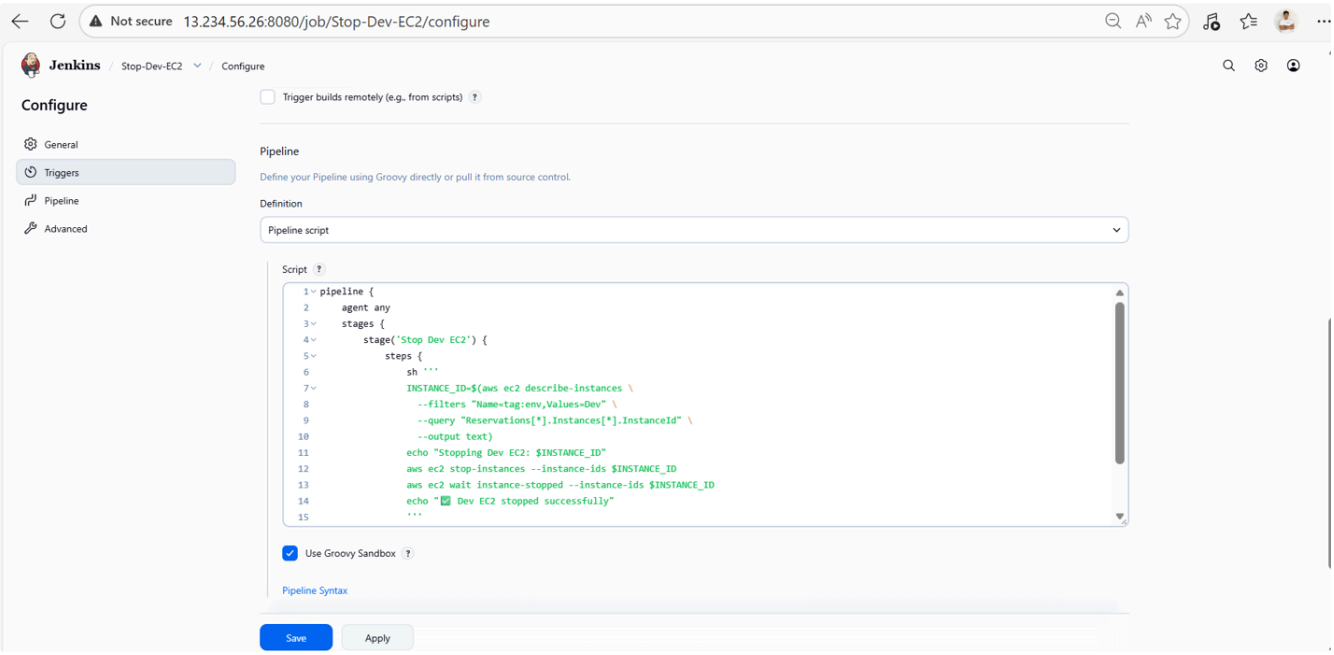
Execution Verification

- Jenkins console output shows stopping process.
- AWS EC2 Console confirms instance state as Stopped.

3.3 Pipeline Design

Declarative pipelines were configured to target instances based on the tag `env=dev`. This ensures the pipeline dynamically finds the correct Instance ID without hardcoding.





3.4 Execution Results

The pipelines were successfully executed, verified by the console outputs showing the transition from `stopped` to `running` (Build #4) and back to `stopped` (Build #1).

Jenkins / Start-Dev-EC2 / #4

- Status
- Changes
- Console Output
- Edit Build Information
- Delete build '#4'
- Timings
- Pipeline Overview
- Restart from Stage
- Replay
- Pipeline Steps
- Workspaces
- Previous Build

Console

```
Started by user Vraj Lalwala
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Start-Dev-EC2
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Start Dev EC2)
[Pipeline] sh
++ aws ec2 describe-instances --filters Name=tag:env,Values=Dev --query 'Reservations[*].Instances[*].InstanceId' --output text
+ INSTANCE_ID=i-0386c901e74fe8adc
+ echo 'Starting Dev EC2: i-0386c901e74fe8adc'
Starting Dev EC2: i-0386c901e74fe8adc
+ aws ec2 start-instances --instance-ids i-0386c901e74fe8adc
{
    "StartingInstances": [
        {
            "InstanceId": "i-0386c901e74fe8adc",
            "CurrentState": {
                "Code": 0,
                "Name": "pending"
            },
            "PreviousState": {
                "Code": 80,
                "Name": "stopped"
            }
        }
    ]
}
+ aws ec2 wait instance-running --instance-ids i-0386c901e74fe8adc
+ echo '✅ Dev EC2 started successfully'
✅ Dev EC2 started successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Jenkins / Stop-Dev-EC2 / #1

- Status
- Changes
- Console Output
- Edit Build Information
- Delete build '#1'
- Timings
- Pipeline Overview
- Restart from Stage
- Replay
- Pipeline Steps
- Workspaces

Console

```
Started by user Vraj Lalwala
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Stop-Dev-EC2
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Stop Dev EC2)
[Pipeline] sh
++ aws ec2 describe-instances --filters Name=tag:env,Values=Dev --query 'Reservations[*].Instances[*].InstanceId' --output text
+ INSTANCE_ID=i-0386c901e74fe8adc
+ echo 'Stopping Dev EC2: i-0386c901e74fe8adc'
Stopping Dev EC2: i-0386c901e74fe8adc
+ aws ec2 stop-instances --instance-ids i-0386c901e74fe8adc
{
    "StoppingInstances": [
        {
            "InstanceId": "i-0386c901e74fe8adc",
            "CurrentState": {
                "Code": 64,
                "Name": "stopping"
            },
            "PreviousState": {
                "Code": 16,
                "Name": "running"
            }
        }
    ]
}
+ aws ec2 wait instance-stopped --instance-ids i-0386c901e74fe8adc
+ echo '✅ Dev EC2 stopped successfully'
✅ Dev EC2 stopped successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Jenkins

- New Item
- Build History

Add description

All +

Build Queue

No builds in the queue.

Build Executor Status

(0 of 2 executors busy)

S	W	Name ↓	Last Success	Last Failure	Last Duration
✓	☼	Start-Dev-EC2	7 min 54 sec #2	N/A	4.7 sec
✓	☼	Stop-Dev-EC2	5 min 1 sec #1	N/A	21 sec

Icon: S M L

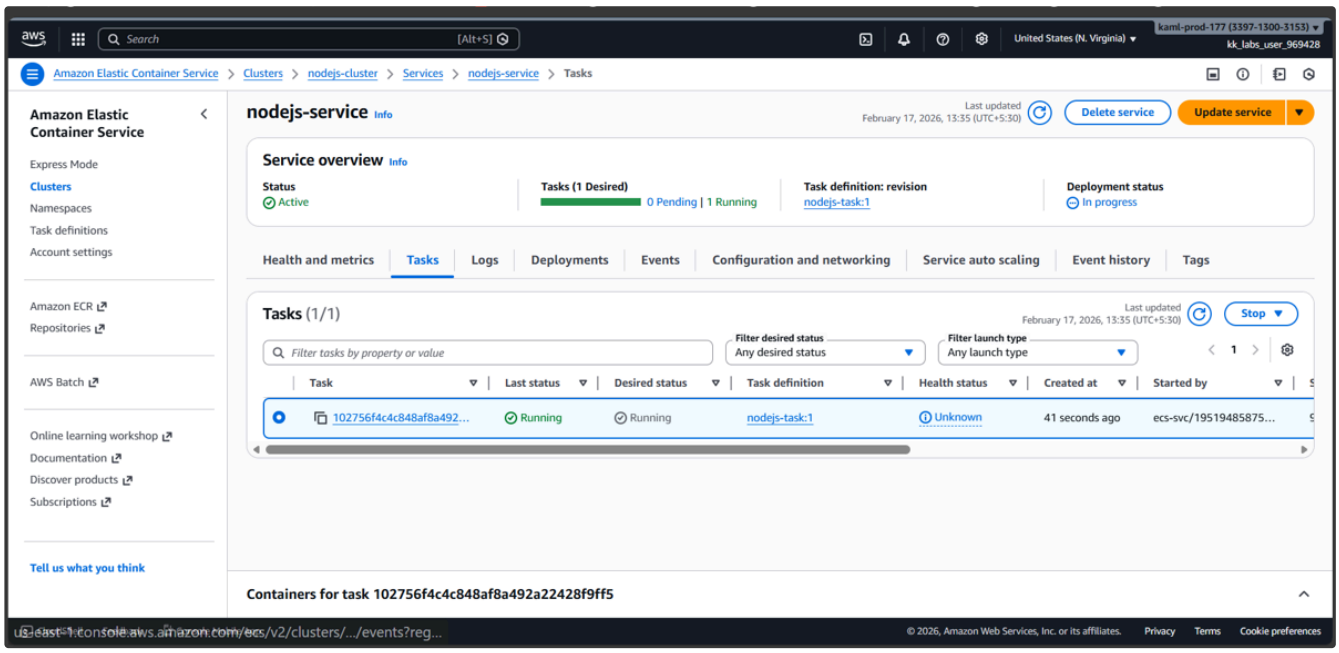
REST API Jenkins 2.541.1

4. Bonus Task: ECS Fargate Automation

4.1 ECS Environment Details

- Cluster Name: nodejs-cluster
- Service Name: nodejs-service
- Task Definition: nodejs-task:1
- Launch Type: Fargate
- Container Image: Stored in Amazon ECR

The ECS service was initially verified in running state.



4.2 Fargate Service Scaling

The automation was extended to ECS Fargate to manage the `nodejs-task-service-0u5dxfs9`. Unlike EC2, "stopping" Fargate involves scaling the `desired-count` to 0.

- **Start ECS Service:** Sets `desired-count` to 1.
- **Stop ECS Service:** Sets `desired-count` to 0.

4.3 Pipeline: Start-ECS-Service

Purpose

Start ECS service by setting desired count to 1.

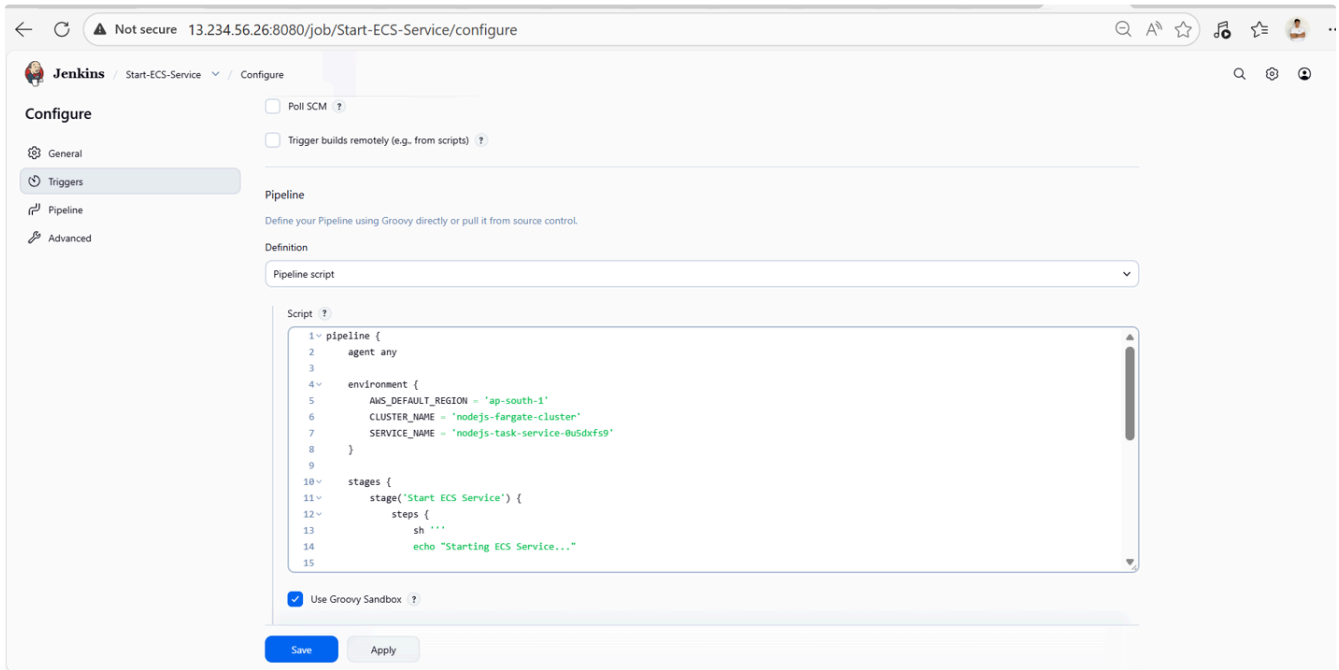
Pipeline Script

```
pipeline {
    agent any
    stages {
```

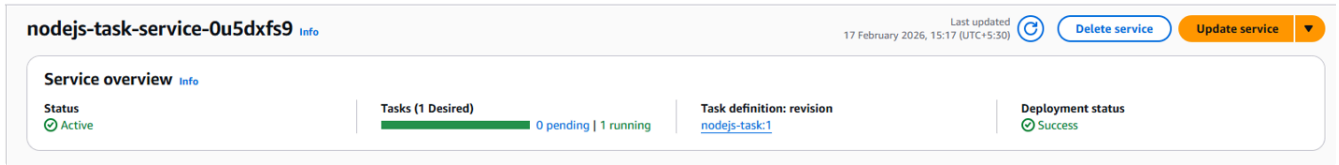
```
        stage('Start ECS Service') {
            steps {
                sh '''
                aws ecs update-service \
                    --cluster nodejs-cluster \
                    --service nodejs-service \
                    --desired-count 1
                aws ecs wait services-stable \
                    --cluster nodejs-cluster \
                    --services nodejs-service
                echo "ECS Service started successfully"
                '''
            }
        }
    }
}

```

Execution Verification

- Jenkins console shows service stabilization.
- ECS Console shows:
  - Desired Count = 1
  - Running Tasks = 1



The service status in the AWS Console confirmed the state changes, showing 1 running tasks after the "Start" pipeline execution.



4.4 Pipeline: Stop-ECS-Service

Purpose

Stop ECS service by setting desired count to 0.

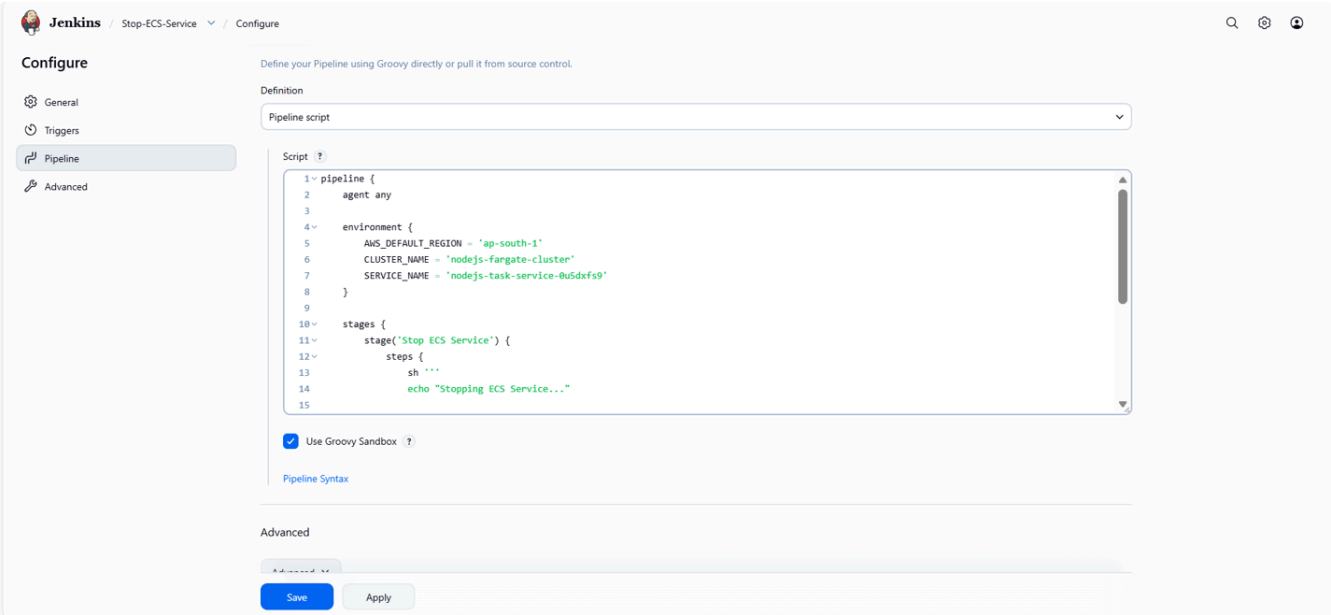
Pipeline Script

```
pipeline {
    agent any
    stages {
        stage('Stop ECS Service') {
            steps {
                sh '''
                aws ecs update-service \
                    --cluster nodejs-cluster \
                    --service nodejs-service \
```

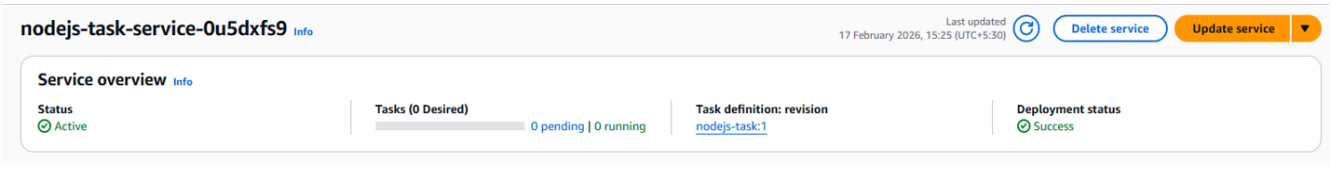
```
                    --desired-count 0
                aws ecs wait services-stable \
                    --cluster nodejs-cluster \
                    --services nodejs-service
                echo "ECS Service stopped successfully"
                '''
            }
        }
    }
}
}

```

Execution Verification

- Jenkins console confirms service stabilization.
- ECS Console shows:
  - Desired Count = 0
  - Running Tasks = 0



The service status in the AWS Console confirmed the state changes, showing 0 running tasks after the "Stop" pipeline execution.



5. Validation Summary

Component	Start Verified	Stop Verified
EC2 Dev Instance	Yes	Yes
ECS Fargate Service	Yes	Yes
IAM Role Attachment	Verified	Not Applicable
Jenkins Pipelines	Successful	Successful

6. Final Outcome

All tasks, including the bonus, were completed successfully. The system now allows for complete environment management (EC2 and Containers) from a single Jenkins dashboard, resulting in significant potential cost savings.